

SPECIFIKACIJA ZALEDNIH FUNKCIONALNOSTI

Aplikacija: Evidenca knjig v knjižnici

Sled sprememb dokumenta			
Verzija	Datum	Avtor	Opis sprememb
1.00	07.10.2026	<i>Dragan Jovanović</i>	Prva verzija dokumenta.
1.01	09.10.2026	<i>Dragan Jovanović</i>	Dodani indexi in spremenjena velikost polj naslov in avtor

1 Objekti

- *Opomba glede primarnih ključev (id):*
Vsi primarni ključi (id) so definirani kot BIGINT IDENTITY(1,1), kar pomeni, da jih baza podatkov generira samodejno. Pri vnosu novih zapisov (npr. NovaKnjiga) se polje id ne pošilja v vhodnem objektu, zaledje pa generirani id vrne v izhodnem odgovoru (KnjigaDTO).

1.1 Vhodni objekti (Request Body DTOs)

1.1.1 NovaKnjiga (Vhodni objekt za ustvarjanje nove knjige)

- isbn (VARCHAR(13), obvezno)
- naslov (NVARCHAR(255), obvezno)
- status_knjige (TINYINT, obvezno – dovoljeni vrednosti 1 "na voljo" ali 3 "ni na voljo")
- avtor (NVARCHAR(255), obvezno)
- zavr_id (TINYINT, opcijsko – šifra iz šifranta žanrov)

1.1.2 PosodobitevKnjiga (Vhodni objekt za urejanje obstoječe knjige)

- isbn (VARCHAR(13), obvezno)
- naslov (NVARCHAR(255), obvezno)
- status_knjige (TINYINT, opcijsko – ročna sprememba dovoljena le na 3 "ni na voljo" ali ohranitev obstoječega)
- avtor (NVARCHAR(255), obvezno)
- zavr_id (TINYINT, opcijsko – šifra iz šifranta žanrov)

1.1.3 NovClan (Vhodni objekt za vnos novega člana)

- ime (NVARCHAR(50), obvezno)
- priimek (NVARCHAR(50), obvezno)
- naslov (NVARCHAR(255), opcijsko)
- telefon (NVARCHAR(50), opcijsko)

1.1.4 Zahtevalzposoja (Vhodni objekt za izvedbo izposoje)

- id_knjige (BIGINT, obvezno)
- id_clana (BIGINT, obvezno)

1.1.5 ZahtevaVracilo (Vhodni objekt za izvedbo vračila)

- id_knjige (BIGINT, obvezno)

1.2 Izhodni objekti (Response DTOs)

1.2.1 KnjigaDTO (Izhodni objekt za vrnjene podatke o knjigi)

- id (BIGINT, obvezno – generiran s strani baze)
- isbn (VARCHAR, obvezno)
- naslov (NVARCHAR, obvezno)
- status_knjige (TINYINT, obvezno)

- status_naziv (NVARCHAR, obvezno – besedilni opis statusa za prikaz, npr. "na voljo")
- avtor (NVARCHAR, obvezno)
- zanr_id (TINYINT, opcijsko)
- zanr_naziv (NVARCHAR, opcijsko – besedilni opis žanra, npr. "Roman")

1.2.2 Clan (Izhodni objekt za podatke o članu)

- id (BIGINT, obvezno – generiran s strani baze)
- ime (NVARCHAR, obvezno)
- priimek (NVARCHAR, obvezno)
- naslov (NVARCHAR, opcijsko)
- telefon (NVARCHAR, opcijsko)

1.2.3 PregledVrsticaDTO (Izhodni objekt za prikaz v tabeli "Pregled")

- id (BIGINT, obvezno)
- isbn (VARCHAR, obvezno)
- naslov (NVARCHAR, obvezno)
- status_knjige (NVARCHAR, obvezno – besedilni naziv statusa, npr. "na voljo")
- avtor (NVARCHAR, obvezno)
- zanr (NVARCHAR, opcijsko – besedilni naziv žanra)
- datum_izposoje (VARCHAR, opcijsko)
- dni_izposoje (VARCHAR, opcijsko)
- clan_id (VARCHAR, opcijsko)
- clan (NVARCHAR, opcijsko – ime in priimek člana)

1.2.4 LogIzposoje (Izhodni objekt za revizijsko sled)

- cas (DATETIME)
- akcija (VARCHAR)
- izposoja_id (BIGINT)
- knjiga_id (BIGINT)
- clan_id (BIGINT)
- datum_izposoje (DATE)
- datum_vracila (DATE)
- naslov_knjige (NVARCHAR)
- naziv_clana (NVARCHAR)
- naslov_clana (NVARCHAR)
- uporabnik (NVARCHAR)

1.2.5 StatusKnjige (Izhodni objekt za šifrant statusov)

- id (TINYINT/INT)
- naziv (NVARCHAR)
- za_izposajo (BIT)
- za_vracilo (BIT)

1.2.6 ZanrKnjige (Izhodni objekt za šifrant žanrov)

- id (TINYINT/INT)
- naziv (NVARCHAR)

2 Opis

Ta specifikacija opredeljuje zaledne funkcionalnosti (Backend API) ter podatkovne strukture za aplikacijo Evidenca knjig v knjižnici. Zaledni sistem omogoča upravljanje knjižničnega gradiva (vnos, posodabljanje in iskanje knjig), izvedbo procesov izposoje in vračila ter vpogled v revizijsko sled (log dogodkov) in zbirni pregled stanja.

V okviru projektiranja je bil v celoti razvit podatkovni model ter v bazi že implementirana celotna T-SQL logika (viewi, triggerji in stored procedure). Backend razvijalec mora preko API točk le klicati pripravljene T-SQL objekte.

V trenutnem prototipu ni predvideno ažuriranje šifrantov, kot tudi ne članov. Za potrebe testa se jih lahko vnese ali uvozi s pomočjo SQL skripte direktno v bazo.

Ključne značilnosti in poslovna pravila zalednega sistema:

- **Obvezni podatki:** Pri vseh operacijah vnosa in posodabljanja knjig (POST /api/knjige, PUT /api/knjige/{id}) so polja ISBN, naslov knjige, avtor in status knjige striktno obvezna (NOT NULL).
- **Generiranje primarnih ključev:** Vsi primarni ključi (id) v bazi podatkov so definirani kot BIGINT IDENTITY(1,1). Primarni ključi šifrantov so definirani kot TINYINT IDENTITY(1,1). Baza podatkov samodejno dodeli unikatni id ob vnosu novih zapisov (npr. ob ustvarjanju nove knjige), zato se ta vrednost ne pošilja v vhodnih zahtevah odjemalca.
- **Avtomatska poslovna logika v T-SQL:** Celotno upravljanje stanj knjig, preverjanje razpoložljivosti in osveževanje statusov se izvaja neposredno na nivoju baze podatkov. Stored proceduri (SP_izposoja, SP_vracilo) izvedeta le osnova vnosa oz. posodobitve v tabeli dbo.izposoja, medtem ko vso preverjanje pogojev, validacijo in dejansko spremembo statusov v tabeli dbo.knjige avtomatsko prevzame ta podrejena INSTEAD OF sprožilca (trg_PreveriStatusKnjige in trg_SpremeniStatusObVracilu). Backend API tako le sproži klic ustrezne shranjene procedure ali tabele.
- **Revizijska sled in viewi:** Baza samodejno beleži vse dogodke izposoj in vračil v tabelo log_izposoje (prek audit triggerjev). Prikaz celotne revizijske sledi (GET /api/izposoje/log) poteka kronološko brez filtriranja. Združeni podatki o knjigah, trenutnih izposojah in članih se pridobivajo neposredno iz SQL view-a dbo.pregled.

3 Seznam metod

Ime metode	Kratek opis
Iskanje in pregled knjig	<i>GET /api/knjige</i> Iskanje in pregled knjig glede na vnesena URL parametra (isbn, naslov).
Vnos nove knjige	<i>POST /api/knjige</i> Vnos nove knjige v sistem. Baza samodejno preveri veljavnost statusa in dodeli id.
Urejanje podatkov knjige	<i>PUT /api/knjige/{id}</i> Posodobitev podatkov obstoječe knjige. Preveri dovoljene spremembe statusa preko triggerja.
Pregled članov knjižnice	<i>GET /api/clani</i> Vrne seznam vseh registriranih članov knjižnice.
Vnos novega člana	<i>POST /api/clani</i> Vnos novega člana knjižnice v sistem.
Izposoja knjige	<i>POST /api/izposoje/izposodi</i> Izvedba izposoje knjige (kliče stored proceduro SP_izposoja).
Vračilo knjige	<i>POST /api/izposoje/vrni</i> Izvedba vračila knjige (kliče stored proceduro SP_vracilo).
Pregled stanja knjig	<i>GET /api/pregled</i> Vrne zbirni pregled stanja knjig neposredno iz SQL view-a dbo.pregled.
Pregled revizijske sledi	<i>GET /api/izposoje/log</i> Prikaz celotne kronološke revizijske sledi iz tabele log_izposoje.
Šifrant statusov	<i>GET /api/sifrant/statusi</i> Vrne šifrant vseh možnih statusov knjig.
Šifrant žanrov	<i>GET /api/sifrant/zanri</i> Vrne šifrant vseh žanrov.

4 Metode

4.1 GET /api/knjige

Opis metode: Pridobivanje seznama knjig z možnostjo filtriranja po ISBN in/ali naslovu knjige. Metoda omogoča iskanje po delnem besedilu naslova ter vrača celotne podatke o knjigah skupaj z besedilnimi nazivi statusov in žanrov.

4.1.1 Vhodni podatki

URL parametri (Query parameters):

- isbn (VARCHAR(13), opcijsko)
- naslov (NVARCHAR(255), opcijsko)

4.1.2 Izhodni podatki

Seznam (array) objektov KnjigaDTO:

- id (BIGINT): Primarni ključ knjige.
- isbn (VARCHAR): Številka ISBN.
- naslov (NVARCHAR): Naslov knjige.
- status_knjige (TINYINT): Numerična šifra statusa.
- status_naziv (NVARCHAR): Besedilni naziv statusa za prikaz (npr. "na voljo").
- avtor (NVARCHAR): Ime in priimek avtorja.
- zanr_id (TINYINT, opcijsko): Šifra žanra.
- zanr_naziv (NVARCHAR, opcijsko): Besedilni naziv žanra (npr. "Roman").

4.1.3 Logika

Zaledni sistem izvede SQL poizvedbo nad tabelo dbo.knjige. Ker je polje status_knjige obvezno (NOT NULL), se s tabelo dbo.status izvede obvezna povezava (INNER JOIN). Za pridobitev naziva žanra se uporabi neobvezna povezava (LEFT JOIN) s tabelo dbo.zanr, saj je zanr_id opcijsko polje in knjiga morda nima dodeljenega žanra. Če sta navedena parametra isbn ali naslov, se v pogoju WHERE izvede filtriranje s pomočjo operatorja LIKE.

```
SELECT
    k.id,
    k.isbn,
    k.naslov,
    k.status_knjige,
    s.naziv AS status_naziv,
    k.avtor,
    k.zanr_id,
    z.naziv AS zanr_naziv
FROM dbo.knjige k
INNER JOIN dbo.status s ON k.status_knjige = s.id
LEFT JOIN dbo.zanr z ON k.zanr_id = z.id
WHERE (@isbn IS NULL OR k.isbn LIKE '%' + @isbn + '%')
    AND (@naslov IS NULL OR k.naslov LIKE '%' + @naslov + '%')
ORDER BY k.id DESC;
```

4.2 POST /api/knjige

Opis metode: Vnos nove knjige v sistem. Baza podatkov ob vnosu samodejno generira unikatni identifikator id. Metoda vrne novoustvarjeni objekt KnjigaDTO z vsemi pripadajočimi podatki in besedilnimi nazivi.

4.2.1 Vhodni podatki

Telo zahteve (Request Body) – objekt NovaKnjiga:

- isbn (VARCHAR(13), obvezno): Številka ISBN.
- naslov (NVARCHAR(255), obvezno): Naslov knjige.
- status_knjige (TINYINT, obvezno): Šifra statusa (dovoljeni vrednosti: 1 za "na voljo" ali 3 za "ni na voljo").
- avtor (NVARCHAR(255), obvezno): Ime in priimek avtorja.
- zanr_id (TINYINT, opcijsko): Šifra žanra iz šifranta.

4.2.2 Izhodni podatki

Vrnen objekt KnjigaDTO (Status 201 Created):

- id (BIGINT): Primarni ključ, ki ga generira baza.
- isbn (VARCHAR): Številka ISBN.
- naslov (NVARCHAR): Naslov knjige.
- status_knjige (TINYINT): Šifra statusa.
- status_naziv (NVARCHAR): Besedilni naziv statusa.
- avtor (NVARCHAR): Ime in priimek avtorja.
- zanr_id (TINYINT, opcijsko): Šifra žanra.
- zanr_naziv (NVARCHAR, opcijsko): Besedilni naziv žanra.

4.2.3 Logika

Zaledni sistem najprej izvede vnos (INSERT) v tabelo dbo.knjige. Takoj po vnosu s pomočjo funkcije SCOPE_IDENTITY() pridobi generirani id. Za sestavo celotnega izhodnega odgovora (KnjigaDTO) nato izvede poizvedbo SELECT nad pravkar vstavljeno vrstico ter preko INNER JOIN (za status) in LEFT JOIN (za žanr) pridobi pripadajoča besedilna naziva.

```
-- 1. Vnos nove knjige
INSERT INTO dbo.knjige (isbn, naslov, status_knjige, avtor, zanr_id)
VALUES (@isbn, @naslov, @status_knjige, @avtor, @zanr_id);
```

```
-- Pridobitev generiranega ID-ja
DECLARE @nov_id BIGINT = SCOPE_IDENTITY();
```

```
-- 2. Vračilo ustvarjenega objekta s pripadajočimi nazivi
SELECT
    k.id,
    k.isbn,
    k.naslov,
    k.status_knjige,
    s.naziv AS status_naziv,
    k.avtor,
    k.zanr_id,
    z.naziv AS zanr_naziv
FROM dbo.knjige k
INNER JOIN dbo.status s ON k.status_knjige = s.id
LEFT JOIN dbo.zanr z ON k.zanr_id = z.id
WHERE k.id = @nov_id;
```

4.3 PUT /api/knjige/{id}

Opis metode: Posodobitev podatkov obstoječe knjige. Metoda omogoča posodobitev naslova, avtorja, ISBN številke, žanra ter statusa knjige. Ob spremembi statusa se veljavnost operacije preveri v bazi, pri čemer je ročna sprememba statusa dovoljena le na vrednost 3 ("ni na voljo") ali ohranitev obstoječega. Namen statusa 3 je da knjigo označimo kot izgubljeno, uničeno ali podobno.

4.3.1 Vhodni podatki

URL parameter (Path parameter):

- id (BIGINT, obvezno): Identifikator knjige za posodobitev.

Telo zahteve (Request Body) – objekt PosodobitevKnjiga:

- isbn (VARCHAR(13), obvezno): Številka ISBN.
- naslov (NVARCHAR(255), obvezno): Naslov knjige.
- status_knjige (TINYINT, opcijsko): Šifra statusa (dovoljena vrednost le 3 ali izpustitev/null za ohranitev obstoječega).
- avtor (NVARCHAR(255), obvezno): Ime in priimek avtorja.
- zanr_id (TINYINT, opcijsko): Šifra žanra iz šifranta.

4.3.2 Izhodni podatki

Vrnjen osvežen objekt KnjigaDTO (Status 200 OK):

- id (BIGINT): Primarni ključ knjige.
- isbn (VARCHAR): Številka ISBN.
- naslov (NVARCHAR): Naslov knjige.
- status_knjige (TINYINT): Šifra statusa.
- status_naziv (NVARCHAR): Besedilni naziv statusa.
- avtor (NVARCHAR): Ime in priimek avtorja.
- zanr_id (TINYINT, opcijsko): Šifra žanra.
- zanr_naziv (NVARCHAR, opcijsko): Besedilni naziv žanra.

4.3.3 Logika

Zaledni sistem najprej izvede posodobitev (UPDATE) zapisa v tabeli dbo.knjige za podani @id. Če parameter @status_knjige v zahtevi ni bil posredovan (NULL), se v bazi ohrani trenutna vrednost polja (ISNULL). Takoj po posodobitvi sistem izvede poizvedbo SELECT nad posodobljeno vrstico ter preko INNER JOIN (za status) in LEFT JOIN (za žanr) vrne osvežen objekt KnjigaDTO.

```
-- 1. Posodobitev podatkov knjige
UPDATE dbo.knjige
SET
    isbn = @isbn,
    naslov = @naslov,
    avtor = @avtor,
    zanr_id = @zanr_id,
    status_knjige = ISNULL(@status_knjige, status_knjige)
WHERE id = @id;
```

```
-- 2. Vračilo osveženega objekta s pripadajočimi nazivi
SELECT
    k.id,
    k.isbn,
    k.naslov,
    k.status_knjige,
    s.naziv AS status_naziv,
    k.avtor,
    k.zanr_id,
    z.naziv AS zanr_naziv
```

```
FROM dbo.knjige k
INNER JOIN dbo.status s ON k.status_knjige = s.id
LEFT JOIN dbo.zanr z ON k.zanr_id = z.id
WHERE k.id = @id;
```

4.4 GET /api/clani

Opis metode: Pridobivanje celotnega seznama vseh članov knjižnice. Metoda ne sprejema vhodnih parametrov ter vrne seznam objektov z osnovnimi osebnimi in kontaktni podatki članov.

4.4.1 Vhodni podatki

Brez parametrov

4.4.2 Izhodni podatki

Seznam (array) objektov Clan:

- id (BIGINT): Primarni ključ člana.
- ime (NVARCHAR): Ime člana.
- priimek (NVARCHAR): Priimek člana.
- naslov (NVARCHAR): Naslov bivališča člana (ulica, hišna številka, kraj).
- telefon (VARCHAR): Telefonska številka člana.

4.4.3 Logika

Zaledni sistem izvede enostavno SQL poizvedbo SELECT nad tabelo dbo.clani brez filtriranja ter vrne vse zapisane člane. Rezultati so razvrščeni po primarnem ključu id naraščajoče.

```
SELECT
    c.id,
    c.ime,
    c.priimek,
    c.naslov,
    c.telefon
FROM dbo.clani c
ORDER BY c.id ASC;
```

4.5 POST /api/clani

Opis metode: Vnos novega člana v sistem knjižnice. Baza podatkov ob vnosu samodejno generira unikatni identifikator id. Metoda vrne novoustvarjeni objekt z vsemi vpisanimi podatki člana.

4.5.1 Vhodni podatki

Telo zahteve (Request Body) – objekt NovClan:

- ime (NVARCHAR(50), obvezno): Ime člana.
- priimek (NVARCHAR(50), obvezno): Priimek člana.
- naslov (NVARCHAR(100), obvezno): Naslov bivališča člana.
- telefon (VARCHAR(20), obvezno): Telefonska številka člana.

4.5.2 Izhodni podatki

Vrnjen objekt Clan (Status 201 Created):

- id (BIGINT): Primarni ključ, ki ga generira baza.
- ime (NVARCHAR): Ime člana.
- priimek (NVARCHAR): Priimek člana.
- naslov (NVARCHAR): Naslov bivališča člana.
- telefon (VARCHAR): Telefonska številka člana.

4.5.3 Logika

Zaledni sistem najprej izvede vnos (INSERT) novih podatkov v tabelo dbo.clani. Takoj po vnosu s pomočjo funkcije SCOPE_IDENTITY() pridobi generirani id. Za sestavo izhodnega odgovora nato izvede poizvedbo SELECT nad pravkar vstavljeno vrstico in vrne celoten objekt ClanDTO.

```
-- 1. Vnos novega člana
INSERT INTO dbo.clani (ime, priimek, naslov, telefon)
VALUES (@ime, @priimek, @naslov, @telefon);
```

```
-- Pridobitev generiranega ID-ja
DECLARE @nov_id BIGINT = SCOPE_IDENTITY();
```

```
-- 2. Vračilo ustvarjenega objekta člana
SELECT
    c.id,
    c.ime,
    c.priimek,
    c.naslov,
    c.telefon
FROM dbo.clani c
WHERE c.id = @nov_id;
```

4.6 POST /api/izposoje/izposodi

Opis metode: Izvedba izposoje knjige za določenega člana. Metoda sproži shranjeno proceduro SP_izposoja, ki poskrbi za preverjanje razpoložljivosti knjige, spremembo statusa knjige ter zapis v tabelo izposoj in revizijsko sled (log_izposoje). Podrobno delovanje je opisano v poglavju 5.

4.6.1 Vhodni podatki

Telo zahteve (Request Body) – objekt Zahtevalzposoja:

- Id_knjige (BIGINT, obvezno): Identifikator knjige, ki se izposoja.
- Id_clana (BIGINT, obvezno): Identifikator člana, ki si knjigo izposoja.

4.6.2 Izhodni podatki

- HTTP status 200 OK (brez telesa odgovora / prazno telo) ob uspešni zabeležbi izposoje.
- HTTP status 400 Bad Request, če knjiga ni na voljo za izposajo ali so podatki neveljavni.

4.6.3 Logika

Zaledni sistem pokliče shranjeno proceduro dbo.SP_izposoja in ji predaja vhodna parametra @id_knjige in @id_clana. Shranjena procedura unutar transakcije preveri, ali je knjiga na voljo (status_knjige = 1). Če je, spremeni status knjige na "izposojeno" (status_knjige = 2), vstavi zapis v tabelo izposoj in zapiše dogodek v log.

```
-- Klic shranjene procedure za izposajo knjige
EXEC dbo.SP_izposoja @id_knjige, @id_clana;
```

4.7 POST /api/izposoje/vrni

Opis metode: Izvedba vračila izposojene knjige. Metoda sproži shranjeno proceduro SP_vracilo, ki poskrbi za posodobitev datuma vračila, spremembo statusa knjige nazaj na "na voljo" ter zapis v revizijsko sled (log_izposoje).

4.7.1 Vhodni podatki

Telo zahteve (Request Body) – objekt ZahtevaVracilo:

- Id_knjige (BIGINT, obvezno): Identifikator knjige, ki se vrača.

4.7.2 Izhodni podatki

- HTTP status 200 OK (brez telesa odgovora / prazno telo) ob uspešni zabeležbi vračila.
- HTTP status 400 Bad Request, če za navedeno knjigo ni bila najdena aktivna izposoja.

4.7.3 Logika

Zaledni sistem pokliče shranjeno proceduro dbo.SP_vracilo in ji preda vhodni parameter @id_knjige. Shranjena procedura znotraj transakcije preveri, ali obstaja aktivna izposoja za podano knjigo. Če obstaja, posodobi datum vračila, spremeni status knjige na "na voljo" (status_knjige = 1) ter zapiše dogodek v revizijsko sled.

```
-- Klic shranjene procedure za vračilo knjige  
EXEC dbo.SP_vracilo @id_knjige;
```

4.8 GET /api/pregled

Opis metode: Pridobivanje agregiranega pregleda stanja knjig v knjižnici z uporabo SQL pogleda (View) pregled. Metoda vrne podrobne podatke o vsaki knjigi, vključno z njenim trenutnim statusom, žanrom ter podatki o aktivni izposoji (datum izposoje, število dni izposoje in podatki o članu).

4.8.1 Vhodni podatki

Brez parametrov.

4.8.2 Izhodni podatki

Seznam (array) objektov PregledVrsticaDTO (Status 200 OK):

- id (BIGINT): Identifikator knjige.
- isbn (VARCHAR): ISBN številka knjige.
- avtor (NVARCHAR): Avtor knjige.
- naslov (NVARCHAR): Naslov knjige.
- status_knjige (NVARCHAR): Besedilni naziv statusa knjige (npr. "na voljo", "izposojeno").
- zanr (NVARCHAR, opcijsko): Besedilni naziv žanra knjige.
- datum_izposoje (VARCHAR, opcijsko): Datum izposoje v formatu DD.MM.YYYY.
- dni_izposoje (VARCHAR, opcijsko): Število dni, kar je knjiga izposojena.
- clan_id (VARCHAR, opcijsko): Identifikator člana, ki si je knjigo izposodil.
- clan (NVARCHAR, opcijsko): Ime in priimek člana, ki si je knjigo izposodil.

4.8.3 Logika

Zaledni sistem izvede SQL poizvedbo SELECT neposredno nad SQL view-om dbo.pregled. View že vsebuje vso potrebno agregacijsko logiko in združevanje tabel (knjige, status_knjige, zanr, izposoja, clani).

```
SELECT
    p.id,
    p.isbn,
    p.avtor,
    p.naslov,
    p.status_knjige,
    p.zanr,
    p.datum_izposoje,
    p.dni_izposoje,
    p.clan_id,
    p.clan
FROM dbo.pregled p;
```

4.9 GET /api/izposoje/log

Opis metode: Pridobivanje seznama zapisov iz revizijske sledi (log_izposoje). Metoda omogoča opcijsko filtriranje revizijskih dogodkov glede na identifikator knjige ali identifikator člana.

4.9.1 Vhodni podatki

URL parametri (Query parameters):

- knjiga_id (BIGINT, opcijsko): Filtriranje revizijskih zapisov po ID-ju knjige.
- clan_id (BIGINT, opcijsko): Filtriranje revizijskih zapisov po ID-ju člana.

4.9.2 Izhodni podatki

Seznam (polje) objektov LogIzposoje (Status 200 OK):

- cas (DATETIME): Čas vpisa revizijskega dogodka.
- akcija (VARCHAR): Vrsta operacije (npr. "INSERT", "UPDATE").
- izposoja_id (BIGINT): Identifikator zapisa izposoje.
- knjiga_id (BIGINT): Identifikator knjige.
- clan_id (BIGINT): Identifikator člana.
- datum_izposoje (DATE): Datum izposoje.
- datum_vracila (DATE, opcijsko): Datum vračila knjige.
- naslov_knjige (NVARCHAR, opcijsko): Naslov knjige v trenutku zapisa.
- naziv_clana (NVARCHAR, opcijsko): Ime in priimek člana v trenutku zapisa.
- naslov_clana (NVARCHAR, opcijsko): Naslov člana v trenutku zapisa.
- uporabnik (NVARCHAR, opcijsko): Uporabnik ali sistem, ki je izvedel operacijo.

4.9.3 Logika

Zaledni sistem izvede SQL poizvedbo nad tabelo dbo.log_izposoje. Če sta posredovana parametra @knjiga_id ali @clan_id, se v pogoju WHERE izvede filtriranje po ustreznem polju. Rezultati so razvrščeni po času nastanka dogodka padajoče (najnovejši zapisi na vrhu).

```
SELECT
    l.cas,
    l.akcija,
    l.izposoja_id,
    l.knjiga_id,
    l.clan_id,
```

```

        l.datum_izposoje,
        l.datum_vracila,
        l.naslov_knjige,
        l.naziv_clana,
        l.naslov_clana,
        l.uporabnik
FROM dbo.log_izposoje l
WHERE (@knjiga_id IS NULL OR l.knjiga_id = @knjiga_id)
      AND (@clan_id IS NULL OR l.clan_id = @clan_id)
ORDER BY l.cas DESC;

```

4.10 GET /api/sifrant/statusi

Opis metode: Pridobivanje šifranta vseh možnih statusov knjig v sistemu. Metoda vrne seznam statusov skupaj z njihovimi lastnostmi glede možnosti izposoje in vračila.

4.10.1 Vhodni podatki

Brez parametrov.

4.10.2 Izhodni podatki

Seznam (array) objektov StatusKnjige (Status 200 OK):

- id (INTEGER): Primarni ključ statusa.
- naziv (NVARCHAR): Besedilni naziv statusa (npr. "na voljo", "izposojeno", "ni na voljo").
- za_izposajo (BOOLEAN, opcijsko): Indikator, ali status omogoča izposajo knjige.
- za_vracilo (BOOLEAN, opcijsko): Indikator, ali status omogoča vračilo knjige.

4.10.3 Logika

Zaledni sistem izvede enostavno SQL poizvedbo SELECT nad tabelo dbo.status_knjige ter vrne vse definirane statusse razvrščene po primarnem ključu.

```

SELECT
    s.id,
    s.naziv,
    s.za_izposajo,
    s.za_vracilo
FROM dbo.status s
ORDER BY s.id ASC;

```

4.11 GET /api/sifrant/zanri

Opis metode: Pridobivanje šifranta vseh žanrov knjig, registriranih v knjižničnem sistemu.

4.11.1 Vhodni podatki

Brez parametrov.

4.11.2 Izhodni podatki

Seznam (array) objektov ZanrKnjige (Status 200 OK):

- id (INTEGER): Primarni ključ žanra.
- naziv (NVARCHAR): Besedilni naziv žanra (npr. "roman", "poezija").

4.11.3 Logika

Zaledni sistem izvede SQL poizvedbo SELECT nad tabelo dbo.zanr in vrne vse razpoložljive žanre razvrščene po nazivu naraščajoče.

```
SELECT
    z.id,
    z.naziv
FROM dbo.zanr z
ORDER BY z.naziv ASC;
```

5 T-SQL Logika (Stored Procedure in Triggerji)

Vsi poslovni procesi, preverjanja in revizijske sledi se izvajajo neposredno v bazi T-SQL preko shranjenih postopkov (stored procedure) ter sprožilcev (trigger).

5.1 Stored Procedure

5.1.1 SP_izposoja

- **Namen:** Izvedba procesa izposoje knjige članu knjižnice.
- **Vhodni parametri:** @id_knjige (BIGINT), @id_clana (BIGINT)
- **Delovanje in logika:**
 1. Odpre transakcijo in izvede vnos nove vrstice v tabelo dbo.izposoja s poljema knjiga_id in clan_id ter današnjim datumom (datum_izposoje = CAST(GETDATE() AS DATE)).
 2. Sama procedura ne preverja razpoložljivosti in ne posodablja tabele dbo.knjige neposredno – to samodejno izvede trigger trg_PreveriStatusKnjige (tipa INSTEAD OF INSERT na tabeli dbo.izposoja).
 3. Če trigger ugotovi, da knjiga ni na voljo za izposajo (za_izposajo = 0), sproži napako, kar prekine transakcijo in vrne napako v blok CATCH.
 4. Ob uspešni izvedbi sprožilca in vnosa procedura potrdi transakcijo (COMMIT TRANSACTION).

```
CREATE PROCEDURE [dbo].[SP_izposoja]
    @id_knjige BIGINT,
    @id_clana BIGINT
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        -- Vnos nove izposoje z današnjim datumom
        INSERT INTO izposoja (knjiga_id, clan_id, datum_izposoje, datum_vracila)
        VALUES (@id_knjige, @id_clana, CAST(GETDATE() AS DATE), NULL);

        COMMIT TRANSACTION;
        PRINT 'Izposoja uspešno dodana.';
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;

        THROW;
    END CATCH
END;
```

5.1.2 SP_vracilo

- **Namen:** Izvedba procesa vračila izposojene knjige.
- **Vhodni parametri:** @id_knjige (BIGINT)
- **Delovanje in logika:**
 1. Odpre transakcijo in preveri, ali v tabeli dbo.izposoja obstaja aktivna izposoja za podano knjigo (knjiga_id = @id_knjige AND datum_vracila IS NULL).
 2. Če aktivna izposoja ne obstaja, sproži napako (RAISERROR), prepreči nadaljnje izvajanje in izvede ROLLBACK.

3. Če aktivna izposoja obstaja, posodobi polje datum_vracila = CAST(GETDATE() AS DATE) za to knjigo v tabeli dbo.izposoja.
4. Sama procedura ne posodablja tabele dbo.knjige neposredno – posodobitev statusa knjige nazaj na status "na voljo" samodejno izvede sprožilec trg_SpremeniStatusObVracilu (tipa INSTEAD OF UPDATE na tabeli dbo.izposoja).
5. Ob uspešni posodobitvi procedura potrdi transakcijo (COMMIT TRANSACTION).

```
CREATE PROCEDURE [dbo].[SP_vracilo]
    @id_knjige BIGINT
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        -- 1. Preverimo, ali obstaja aktivna izposoja za to knjigo
        IF NOT EXISTS (
            SELECT 1
            FROM izposoja
            WHERE knjiga_id = @id_knjige AND datum_vracila IS NULL
        )
        BEGIN
            RAISERROR('Za to knjigo ni bila najdena nobena aktivna izposoja.', 16,
1);
        END

        -- 2. Posodobimo datum vračila na današnji datum
        UPDATE izposoja
        SET datum_vracila = CAST(GETDATE() AS DATE)
        WHERE knjiga_id = @id_knjige AND datum_vracila IS NULL;

        COMMIT TRANSACTION;
        PRINT 'Vračilo uspešno zabeleženo.';
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;

        THROW;
    END CATCH
END;
```

5.2 Triggerji

5.2.1 trg_PreveriStatusKnjige

- **Tip:** INSTEAD OF INSERT na tabeli dbo.izposoja
- **Namen:** Validacija razpoložljivosti knjige za izposajo in avtomatska sprememba statusa knjige ob vnosu nove izposoje.
- **Delovanje in logika:**
 1. Preveri, ali imajo knjige med vstavljenimi zapisi (inserted) v tabeli status vrednost za_izposoja = 0 (tj. niso na voljo za izposajo).
 2. Če katera od knjig ni na voljo, sproži napako (RAISERROR), prekliče transakcijo (ROLLBACK TRANSACTION) in prekine izvajanje.
 3. Ob ustreznih razpoložljivosti izvede dejanski vnos v tabelo dbo.izposoja z današnjim datumom (GETDATE()).
 4. Samodejno posodobi status knjige v tabeli dbo.knjige na status, ki predstavlja izposojeno knjigo (za_izposoja = 0 in za_vracilo = 1) v našem šifrantu je to 2.

```

CREATE TRIGGER [dbo].[trg_PreveriStatusKnjige]
ON [dbo].[izposoja]
INSTEAD OF INSERT
AS
BEGIN
    SET NOCOUNT ON;

    -- 1. Preverimo, ali med vstavljenimi vrsticami obstaja kakšna,
    -- ki ima status knjige označen z za_izposoj = 0 (false)
    IF EXISTS (
        SELECT 1
        FROM inserted i
        JOIN knjige k ON i.knjiga_id = k.id
        JOIN status sk ON k.status_knjige = sk.id
        WHERE sk.za_izposoj = 0
    )
    BEGIN
        -- Če knjiga ni na voljo za izposoj, prekličemo operacijo in vržemo napako
        RAISERROR ('Knjiga ni na voljo za izposoj (status ne dovoljuje
izposoje).', 16, 1);
        ROLLBACK TRANSACTION;
        RETURN;
    END

    -- 2. Dodamo nov zapis v izposoj
    INSERT INTO izposoja (knjiga_id, clan_id, datum_izposoje, datum_vracila)
    SELECT knjiga_id, clan_id, GETDATE(), NULL
    FROM inserted i;

    -- 3. Posodobimo status v tabeli knjige na status za "izposojeno"
    UPDATE k
    SET k.status_knjige = (SELECT id FROM status s WHERE s.za_izposoj = 0 AND
s.za_vracilo = 1)
    FROM knjige k
    JOIN inserted i ON k.id = i.knjiga_id;
END;

```

5.2.2 trg_SpremeniStatusObVracilu

- **Tip:** INSTEAD OF UPDATE na tabeli dbo.izposoja
- **Namen:** Avtomatska posodobitev statusa knjige nazaj na "na voljo" ob zabeleženem vračilu.
- **Delovanje in logika:**
 1. Posodobi polji datum_vracila in datum_izposoje v tabeli dbo.izposoja.
 2. Če je določen datum_vracila IS NOT NULL, samodejno posodobi status ustrezne knjige v tabeli dbo.knjige nazaj na status "na voljo" (za_izposoj = 1 in za_vracilo = 0).

```

CREATE TRIGGER [dbo].[trg_SpremeniStatusObVracilu]
ON [dbo].[izposoja]
INSTEAD OF UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    -- 1. Posodobimo samo datum_vracila v tabeli izposoja
    UPDATE i
    SET i.datum_vracila = ins.datum_vracila,
        i.datum_izposoje = ins.datum_izposoje
    FROM izposoja i
    JOIN inserted ins ON i.id = ins.id;

    -- 2. Posodobimo status v tabeli knjige na 'na_voljo'
    UPDATE k
    SET k.status_knjige = (SELECT id FROM status WHERE za_izposoj = 1 AND
za_vracilo = 0)
    FROM knjige k

```

```

        JOIN inserted ins ON k.id = ins.knjiga_id
        WHERE ins.datum_vracila IS NOT NULL; -- Status posodobimo le, če je datum
        vračila določen
    END;

```

5.2.3 Audit Triggerja (TR_izposoja_Audit_Insert in TR_izposoja_Audit_Update)

- **Tip:** AFTER INSERT in AFTER UPDATE na tabeli dbo.izposoja
- **Namen:** Samodejno generiranje nefiltrirane revizijske sledi v tabelo dbo.log_izposoje ob vsakem novem vnosu ali posodobitvi izposoje.
- **Delovanje in logika:**
 1. TR_izposoja_Audit_Insert ob vstavljanju v dbo.izposoja zapiše dogodek z akcijo 'INSERT', ID-jem izposoje, knjige, člana, datumi ter z ustreznim povezovanjem (LEFT OUTER JOIN) pridobi še naslov knjige, ime in priimek ter naslov člana.
 2. TR_izposoja_Audit_Update ob posodobitvi dbo.izposoja na enak način zapiše nov revizijski zapis z akcijo 'UPDATE'.
 3. **Beleženje uporabnika:** Polje uporabnik v tabeli log_izposoje je predvideno za beleženje identitete uporabnika, ki je izvedel akcijo. Ker trenutni prototip aplikacije še ne podpira avtentikacije in prijave uporabnikov, se v to polje začasno zapisuje vrednost 'USER PLACEHOLDER'. Ob bodoči implementaciji sistema za prijavo se bo ta vrednost nadomestila z dinamičnim uporabniškim imenom prijavljenega uporabnika.

```

CREATE TRIGGER [dbo].[TR_izposoja_Audit_Insert]
ON [dbo].[izposoja]
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO [dbo].[log_izposoje] ([akcija], [izposoja_id], knjiga_id, clan_id,
    datum_izposoje, datum_vracila, naslov_knjige, naziv_clana, naslov_clana, uporabnik)
    SELECT
        'INSERT',
        i.[id],
        i.knjiga_id,
        i.clan_id,
        i.datum_izposoje,
        i.datum_vracila,
        k.naslov,
        CONCAT_WS(' ', c.ime, c.priimek),
        c.naslov,
        'USER PLACEHOLDER'

    FROM inserted i
    left outer join knjige k on k.id = i.knjiga_id
    left outer join clan c on c.id = i.clan_id;
END;
GO

CREATE TRIGGER [dbo].[TR_izposoja_Audit_Update]
ON [dbo].[izposoja]
AFTER update
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO [dbo].[log_izposoje] ([akcija], [izposoja_id], knjiga_id, clan_id,
    datum_izposoje, datum_vracila, naslov_knjige, naziv_clana, naslov_clana, uporabnik)
    SELECT
        'UPDATE',
        i.[id],
        i.knjiga_id,
        i.clan_id,
        i.datum_izposoje,

```

```
        i.datum_vracila,  
        k.naslov,  
        CONCAT_WS(' ', c.ime, c.priimek),  
        c.naslov,  
        'USER PLACEHOLDER'  
  
FROM inserted i  
left outer join knjige k on k.id = i.knjiga_id  
left outer join clan c on c.id = i.clan_id;  
END;
```